

Architettura degli Elaboratori

Esercizi sull'architettura CPU: istruzioni e malfunzionamenti



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Andrea Sterbini, Iacopo
Masi, Claudio di Ciccio

[S&PdC]

4.1-4.4



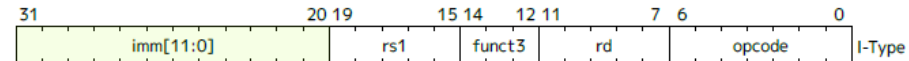
Riepilogo



SAPIENZA
UNIVERSITÀ DI ROMA

Istruzione jalr (Jump relative to Register)

L'istruzione **jalr rs** è di tipo I



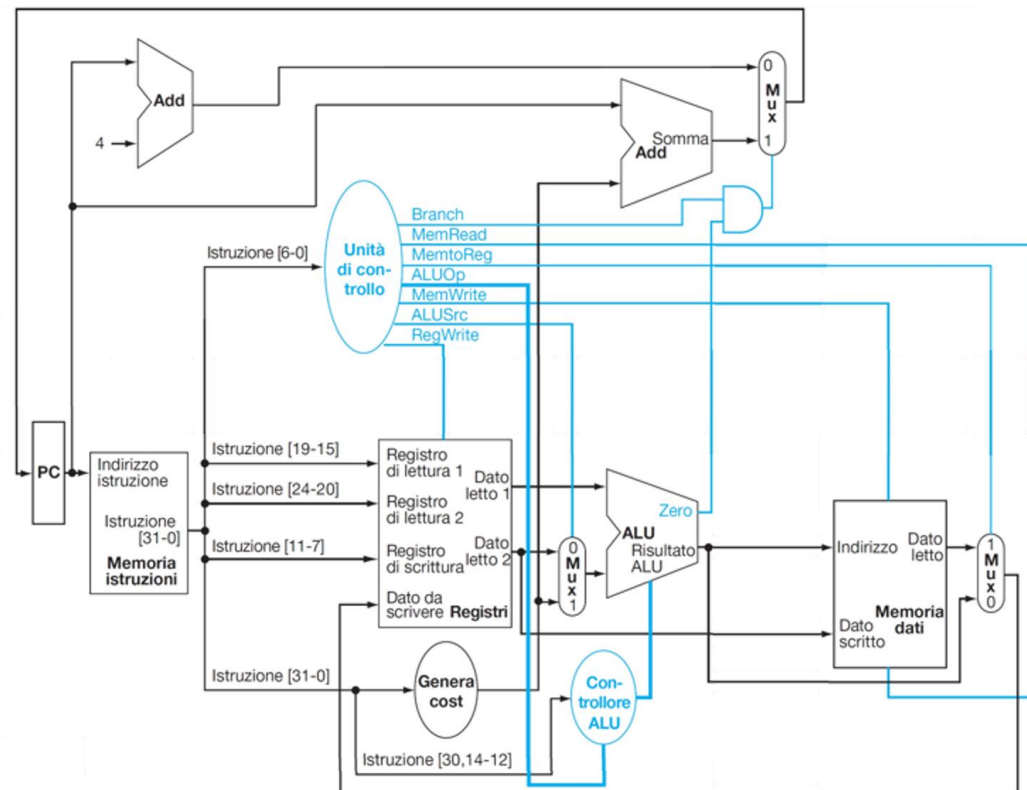
Cosa fa: trasferisce nel **PC** il risultato della **ALU** ($rs + Imm$)

Unità funzionali: **ALU** calcola indirizzo $rs + IMM$

MUX per selezionare il PC dall'uscita della **ALU**

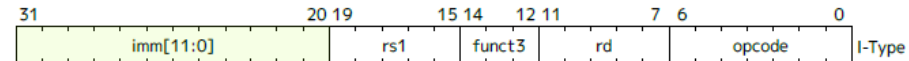
Flusso dei dati: Istruzione[31-0] → Estensione Segno → (Somma con RS) → **MUX** → **PC**

Segnali di controllo: **JumpToReg** che abilita il **MUX** per inserire in **PC** il valore del registro



Istruzione jalr (Jump relative to Register)

L'istruzione **jalr rs** è di tipo I



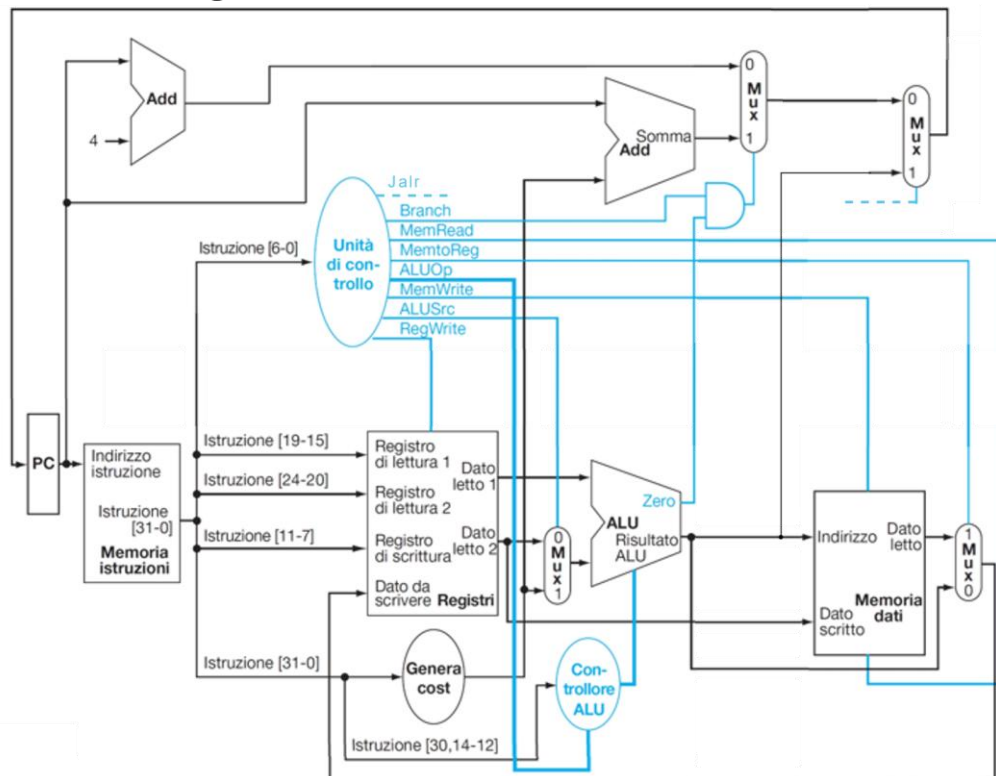
Cosa fa: trasferisce nel **PC** il risultato della **ALU** ($rs + Imm$)

Unità funzionali: **ALU** calcola indirizzo $rs + IMM$

MUX per selezionare il PC dall'uscita della **ALU**

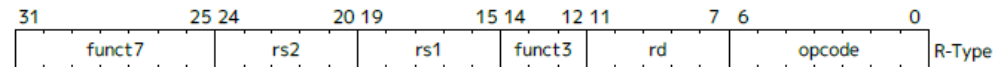
Flusso dei dati: Istruzione[31-0] → Estensione Segno → (Somma con RS) → MUX → PC

Segnali di controllo: **JumpToReg** che abilita il MUX per inserire in PC il valore del registro



Istruzione jr (Jump to Register)

L'istruzione **jr rs** è di tipo R

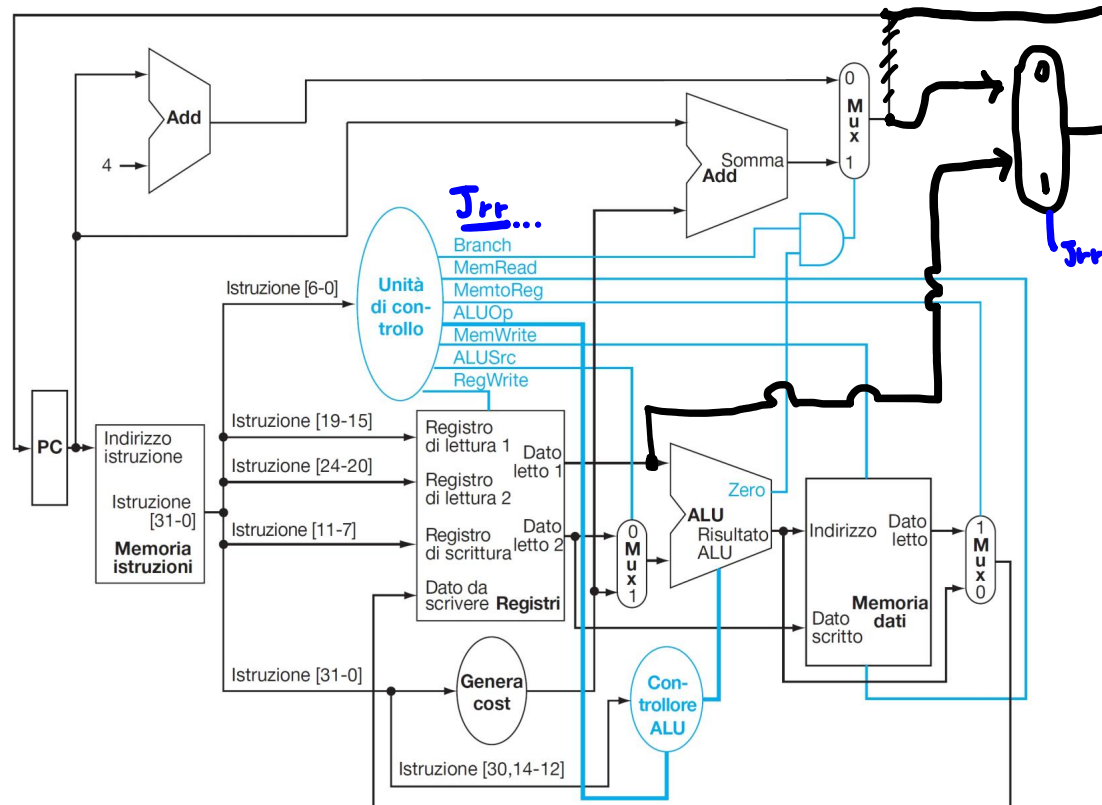


Cosa fa: trasferisce nel **PC** il contenuto del registro **rs**

Unità funzionali: **MUX** per selezionare il PC dall'uscita del blocco registri

Flusso dei dati: **Registri[rs] → PC**

Segnali di controllo: **JumpToReg** che abilita il MUX per inserire in PC il valore del registro



Tempo necessario:
Fetch+Reg

Jump Relative to Register

Aggiungere alla CPU l'istruzione **jrr rs** (Jump Relative to Register)

di tipo R, che salta all'indirizzo (relativo al PC) contenuto nel registro **rs**

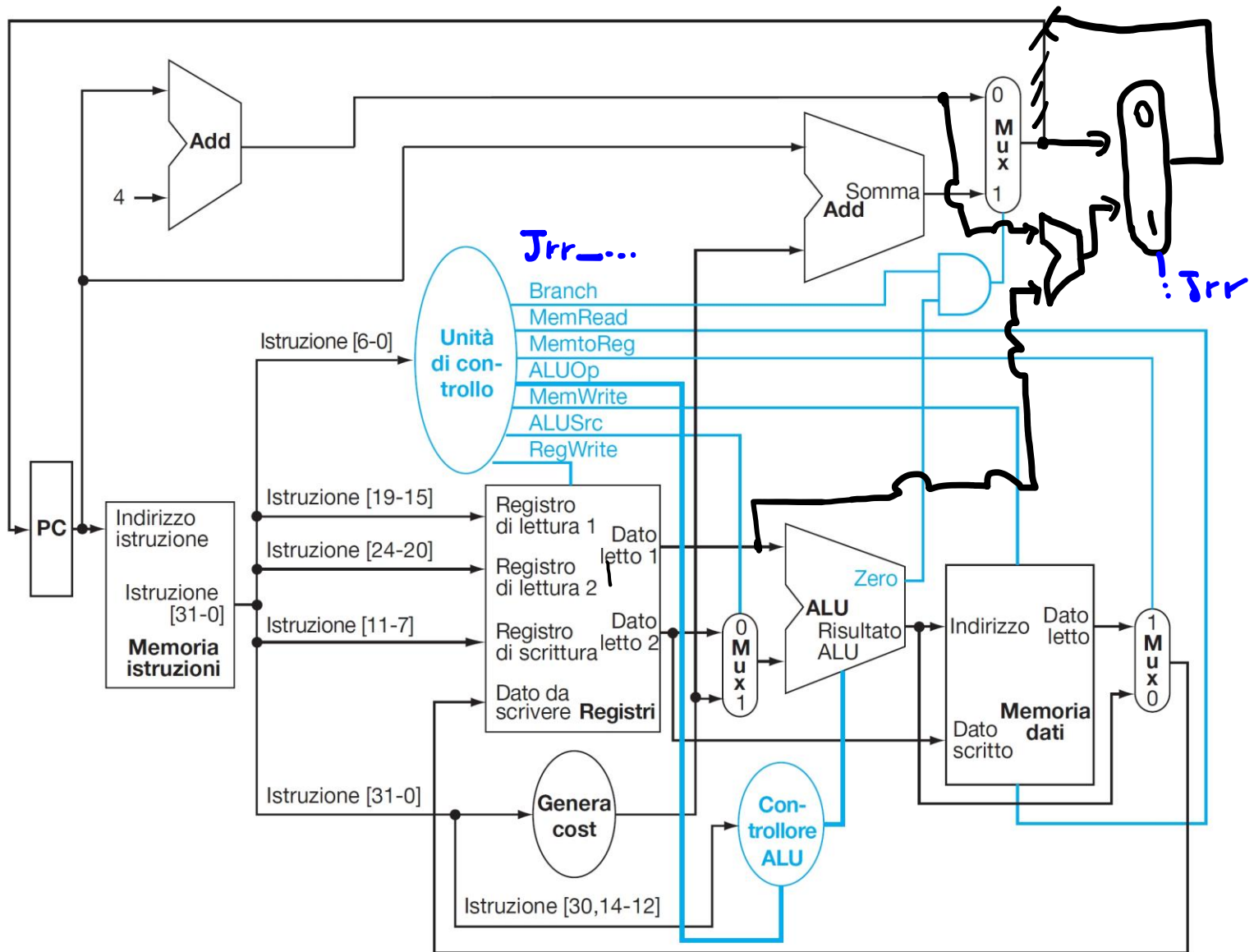
Ovvero che esegue come prossima istruzione quella che si trova all'indirizzo

$PC+4+Registri[rs]$

- a) Modificare lo schema per realizzare l'istruzione (riciclando il più possibile)
- b) Indicare tutti i segnali di controllo che la CU deve generare
- c) Calcolare il tempo di esecuzione della istruzione assumendo che:

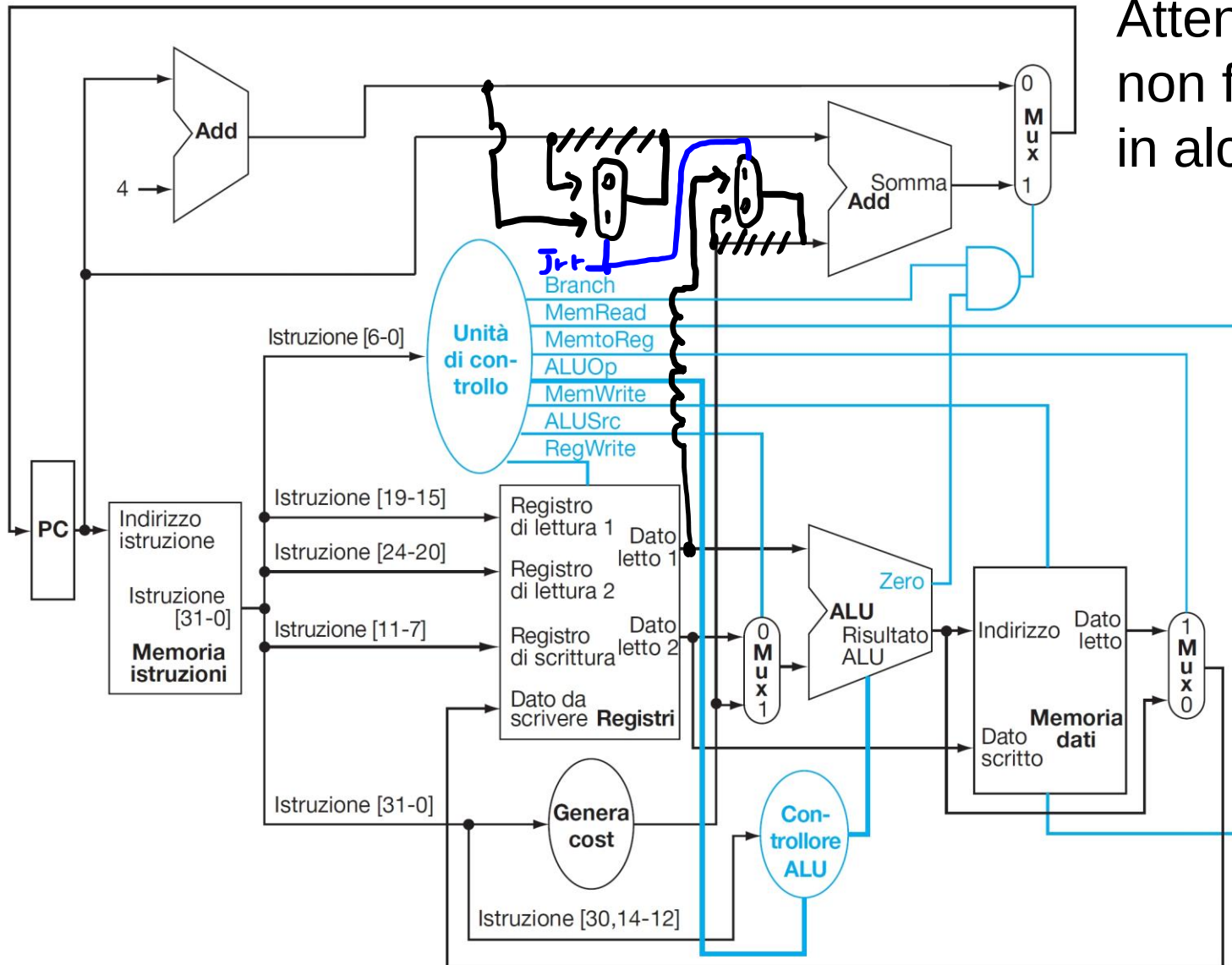
Accesso a memorie = 66ns, accesso ai registri = 33ns, ALU e sommatori = 100ns

jrr PC+4+rs1



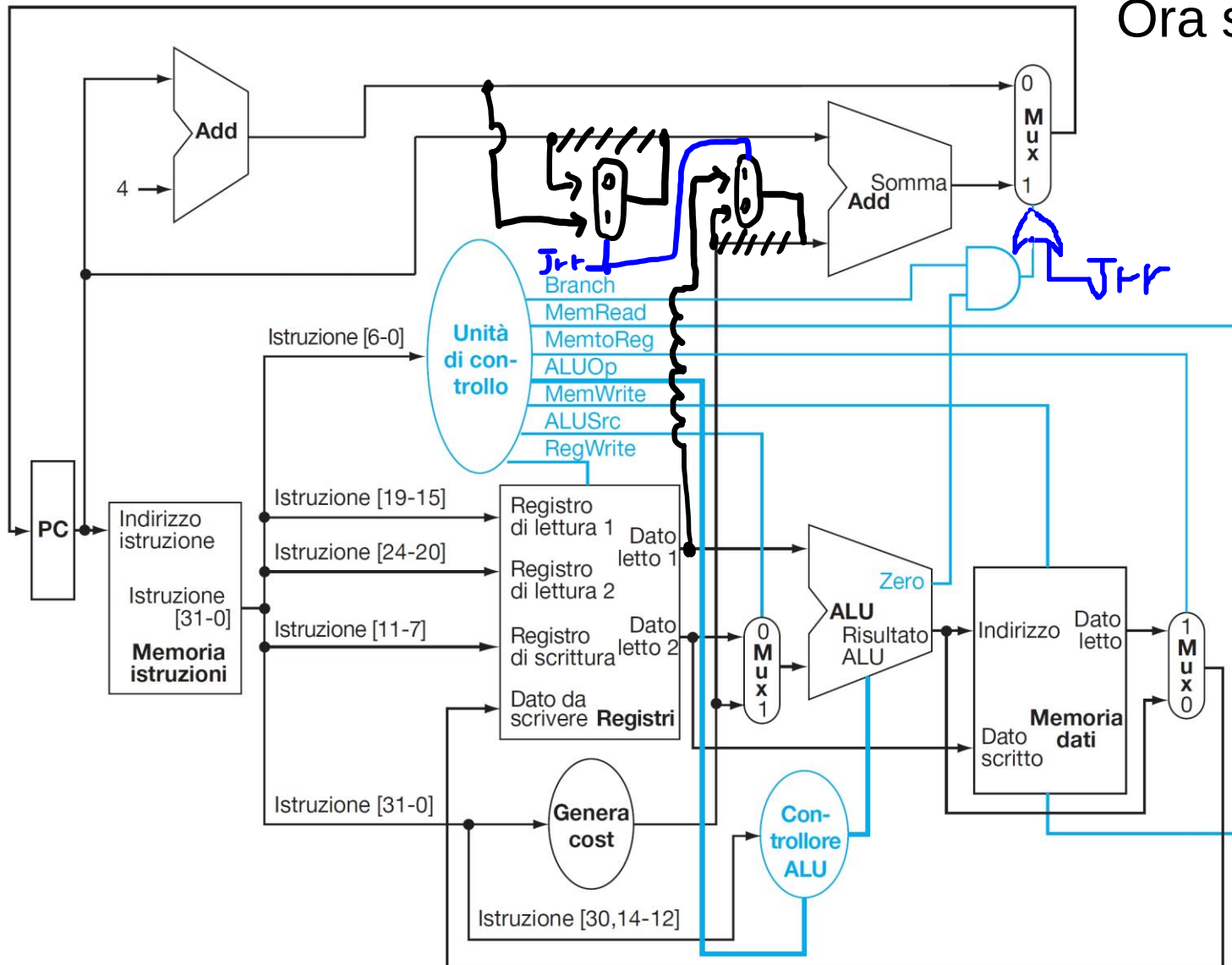
jrr PC+4+rs1 (soluzione più efficiente)

Attenzione!
non funziona
in alcuni casi!



jrr PC+4+rs1 (soluzione più efficiente)

Ora sì



Jrr (soluzione /b, c)

(b) Segnali dalla Control Unit

JRR	Reg Dest	ALU Src	Mem toReg	Reg Write	Mem Read	Mem Write	Branch	ALU Op1	ALU Op0
1	X	X	X	0	0 (X)	0	X	X	X

Accesso a memorie = 66ns, accesso ai registri = 33ns, ALU e sommatori = 100ns

(c) Tempo di esecuzione

Fetch 66ns	Reg[rs] 33ns		
PC+4 100ns		salto: PC+4 + Reg[rs] 100ns	PC ← salto 0ns

Totale: 200ns

Non è necessario incrementare il periodo di clock

Istruzione	Instr. Fetch	Instr. Decode	Execution	MEM	Wr. Back	Totale
add	66	33	100		33	232
lw	66	33	100	66	33	298



Esercizio per casa: vj

Si vuole aggiungere alla CPU l'istruzione vectorised jump (**vj**), di tipo I e sintassi assembly

vj *indice*, *x0*, *vettore*

che salta all'indirizzo contenuto nell'elemento indice-esimo del *vettore* di word. Il secondo registro (*rs2*) non è utilizzato e per questo viene settato a zero.

L'assemblatore inserirà nella parte immediata, i 12 bit di scostamento relativi al program counter per raggiungere l'indirizzo di memoria dell'etichetta *vettore*. In altre parole, l'indirizzo di memoria dell'etichetta è uguale $PC + [20-31]$ (i 12 bit dell'istruzione).

Esempio: se in memoria si è definito staticamente il

vettore: **.word** 0x10008000, 0x10008200, 0x10001088, 0x10008444

e al registro **t0** è assegnato il valore **3** allora

vj *t0*, *vettore*

salterà all'indirizzo **0x10008444** (che è l'elemento con indice 3 del vettore).

- si disegnino le modifiche necessarie a realizzare la funzione, aggiungendo tutti gli eventuali MUX, segnali di controllo, bus, ALU e sommatori (ecc) che ritenete necessari.
- indicate i valori di tutti i segnali di controllo, in modo da eseguire l'istruzione **vj**.
- tenendo conto che il tempo di accesso ai registri (sia in lettura che scrittura) è di **25ns**, l'accesso alla memoria impiega **75ns**, la ALU e i sommatori impiegano **100ns** e ignorando gli altri ritardi, calcolate il tempo di esecuzione minimo della istruzione **vj** e indicate se è necessario aumentare il periodo di clock della CPU per poter svolgere questa nuova istruzione.



Diff and store

Considerare l'architettura RISC-V a ciclo singolo. Si vuole aggiungere l'istruzione di tipo `diff_and_store rs1, rs2, addr`

(calcola la differenza in valore assoluto dei registri `rs1` ed `rs2` e salva il risultato in memoria, all'indirizzo `addr`)

che equivale al seguente frammento di codice:

```
sub rs1, rs1, rs2      # (a)
bge rs1, x0, save      # (b)
sub rs1, x0, rs1       # (c)
save:
    sw rs1, addr       # (d)
```

ossia (a) esegue la sottrazione tra `rs1` ed `rs2`, (b,c) se il risultato è negativo ne cambia il segno, infine (d) salva il valore risultante in memoria all'indirizzo indicato da `addr`. In particolare, `addr` è la parte immediata dell'istruzione e viene usato come indirizzo assoluto di destinazione. L'assemblatore tratterà la parte immediata *addr* come se l'istruzione fosse una `sw`.

Cosa fare se la Control Unit è malfunzionante?



SAPIENZA
UNIVERSITÀ DI ROMA

Possiamo identificare guasti hardware via software?

Control Unit malfunzionante

Segnali di controllo

	ALU Src	Mem toReg	Reg Write	Mem Read	Mem Write	Branch	ALU Op1	ALU Op0
tipo R	0	0	1	X	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	X	1	0	0	0
beq	0	X	0	X	0	1	0	1

Cosa succede se la CU genera dei segnali errati?

Dobbiamo individuare:

- **quale combinazione di segnali venga generata**
- **quali istruzioni vengano influenzate** dalle nuove combinazioni e **cosa facciano**

Una volta individuate le «nuove» funzionalità (fallaci) delle istruzioni...

... possiamo cercare di scrivere un breve programma che evidenzia se la CPU sia malfunzionante o meno

Esempio 1: RegWrite $\leftarrow$ Branch

Supponiamo che:

- il segnale **RegWrite** sia assegnato dal segnale **Branch**
(ossia, se **Branch** = 0 allora **RegWrite** = 0 e se **Branch** = 1 allora **RegWrite**

Assumiamo che:

- MemToReg = 1 solo per la lw ed altrimenti valga 0 (non X)

1) Individuare quali istruzioni ne sono affette e perché

	ALU Src	Mem toReg	Reg Write	Mem Read	Mem Write	Branch	ALU Op1	ALU Op0
tipo R	0	0	0	X	0	0	1	0
lw	1	1	0	1	0	0	0	0
sw	1	0	0	X	1	0	0	0
beq	0	0	1	X	0	1	0	1

Assunzione

Difetto

Esempio 1: RegWrite ← Branch

Si ha il dubbio che, per difetto di progettazione della CU (e.g. cortocircuito):

- il segnale **RegWrite** sia determinato dal segnale **Branch**
(ossia, se **Branch** = 0 allora **RegWrite** = 0 e se **Branch** = 1 allora **RegWrite** = 1)

Si assume che:

- MemToReg sia settato a 0 quando è indeterminato (0 invece che X)
- 1) Individuare quali istruzioni ne sono affette e perché
 - 2) Scrivere un breve programma che scriva in s0 un valore che distingua se la CPU sia correttamente funzionante (s0=1), o malfunzionante (s0=0)

(1)

Le istruzioni affette da questo **guasto** sono:

- tutte le istruzioni che **modificano un registro** (tipo R e lw): lo lasceranno invariato
- **branch**: oltre ad effettuare il salto modificherà uno dei registri
 - il valore scritto sarà il risultato della differenza usata per confrontare i due operandi perché assumiamo MemtoReg=0
 - verrà scritto in un registro corrispondente all'indice Imm[4:1],Imm[11], perché la parte rd è occupata da questa porzione immediata per le istruzioni SB

L'istruzione sw, invece, funzionerà correttamente (perché ha comunque RegWrite a 0)

Esempio 1: RegWrite ← Branch

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

	ALU Src	Mem toReg	Reg Write	Mem Read	Mem Write	Branch	ALU Op1	ALU Op0
tipo R	0	0	0	X	0	0	1	0
lw	1	1	0	1	0	0	0	0
sw	1	0	0	X	1	0	0	0
beq	0	0	1	X	0	1	0	1

Assunzione

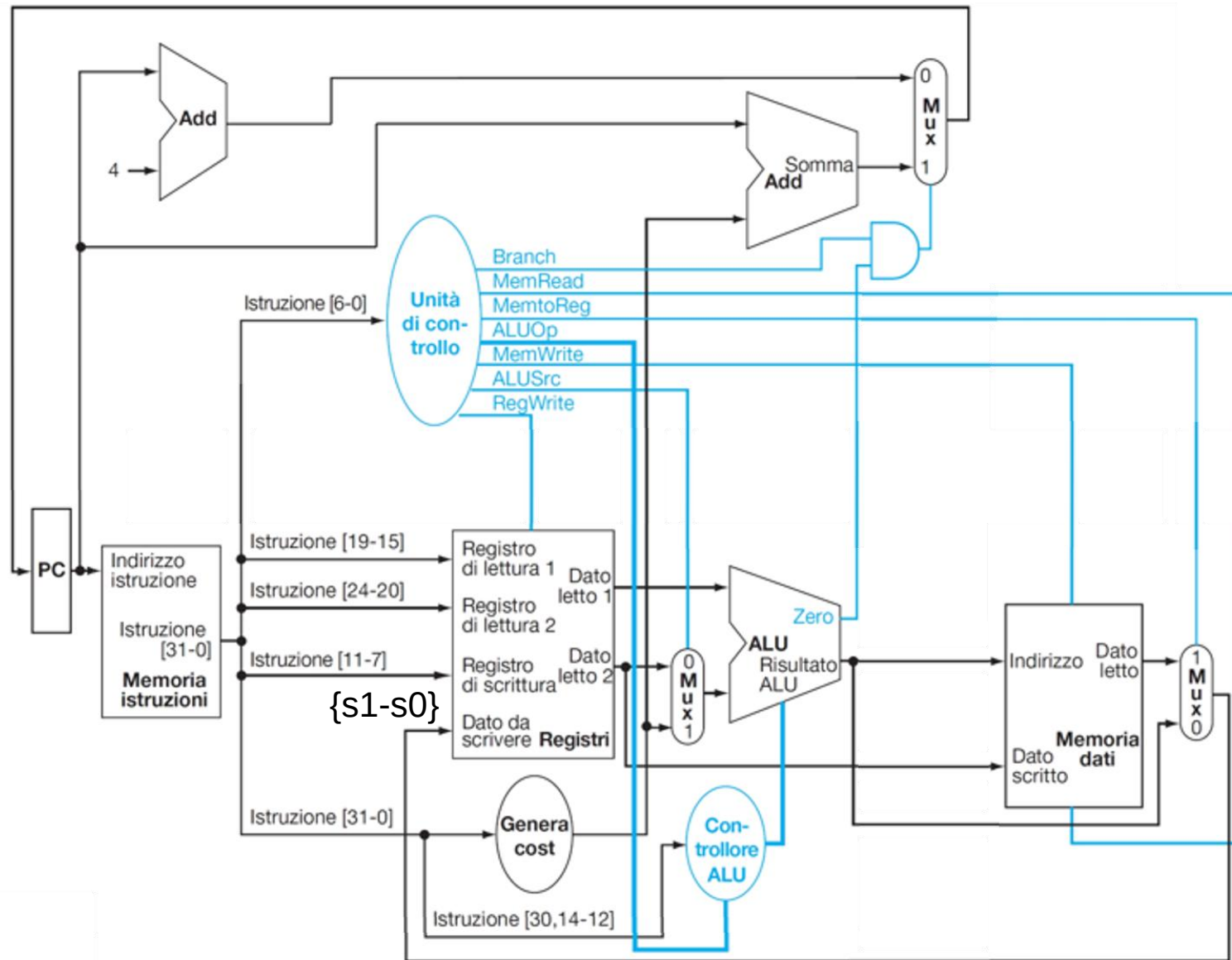
Difetto

Le istruzioni affette da questo **guasto** sono:

- tutte le istruzioni che **modificano un registro** (tipo R e lw): lo lasceranno invariato
- **branch**: oltre ad effettuare il salto modificherà uno dei registri
 - il valore scritto sarà il risultato della differenza usata per confrontare i due operandi perché assumiamo MemtoReg=0
 - verrà scritto in un registro corrispondente all'indice Imm[4:1],Imm[11], perché la parte rd è occupata da questa porzione immediata per le istruzioni SB

L'istruzione sw, invece, funzionerà correttamente (perché ha comunque RegWrite a 0)

RegWrite ← Branch



Individuare il guasto: RegWrite ← Branch

Per individuare se la CPU sia malfunzionante creiamo un programma che lasci il valore 0 nel registro **s0** se lo è, e scriva 1 se funziona correttamente, tenendo conto che:

- Non possiamo caricare un valore in un registro perché RegWrite = 0

Potremmo assumere che in memoria sia presente il valore 1 e provare a leggerlo

<i>.data</i>		<i>.text</i>
<i>uno:</i>	<i>.word</i> 1	<i>main:</i> lw s0, uno

Oppure basta una qualsiasi istruzione che generi un valore diverso da 0 (una **tipo R** o una **li**), come una delle seguenti

<i>su:</i> li s0, 1	# = 1
addi s0, zero, 1	# = 1

Oppure potremmo provare a costruire una **beq** che scriva 0 in un registro se la CPU è rotta, ma non è facile perché non sappiamo dove andrà a scrivere. Inoltre, per distinguere il caso di CPU rotta da quello con CPU funzionante dobbiamo assumere che inizialmente s0=1. DIFFICILE

MemWrite $\leftarrow$ not(RegWrite)

Si ha il dubbio che in alcune CPU RISC-V la Control Unit sia rotta, producendo il segnale di controllo **MemWrite attivo se e solo se NON è attivo il segnale RegWrite**.

Si assume che:

- MemToReg = 1 solo per la lw ed altrimenti valga 0 (mai X)
- a) Indicare quali delle istruzioni funzioneranno male e perché.

MemWrite ← not(RegWrite)

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

	ALU Src	Mem toReg	Reg Write	Mem Read	Mem Write	Branch	ALU Op1	ALU Op0
tipo R	0	0	1	X	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	0	0	X	1	0	0	0
beq	0	0	0	X	1	1	0	1

Assunzione

Difetto

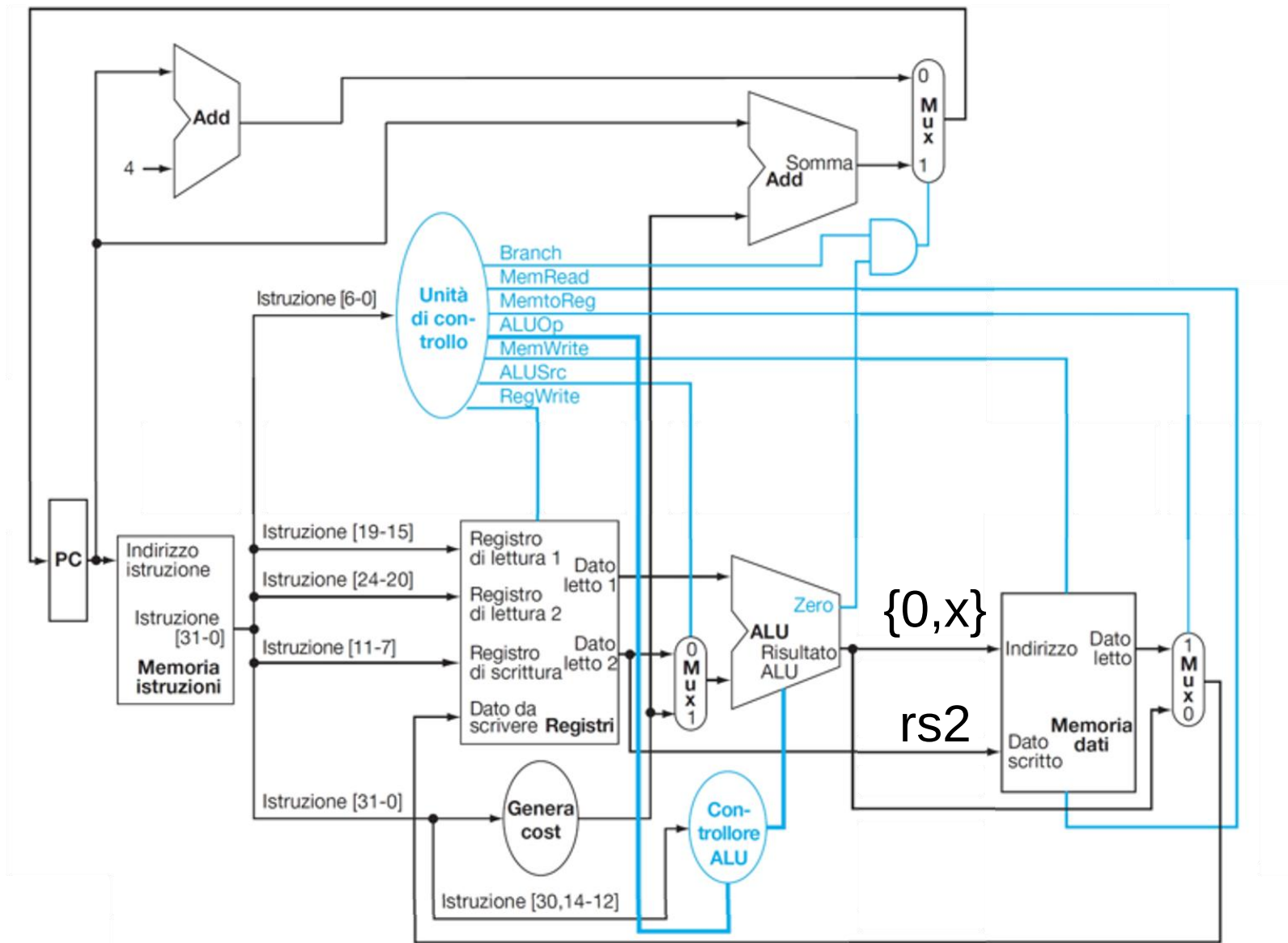
a) «**MemWrite attivo se e solo se NON è attivo il segnale RegWrite**» vuol dire che

- se RegWrite = 0 allora MemWrite = 1
- se RegWrite = 1 allora MemWrite = 0

Quindi sono danneggiate le istruzioni che hanno i due segnali entrambi a 0 o entrambi a 1

- **lw**: funziona correttamente (1, 0)
- **sw**: funziona correttamente (0, 1)
- **beq**: salta correttamente MA INOLTRE SCRIVE IN MEMORIA (0, 1) – invece che (0, 0)
- **tipo R**: funzionano correttamente (1, 0)

MemWrite $\leftarrow$ not(RegWrite)



MemWrite ← not(RegWrite)

b) Per distinguere se la CPU è rotta dobbiamo quindi usare l'istruzione di branch:

beq **rs1**, **rs2**, *etichetta* che memorizza

- il valore del registro rs2 all'indirizzo calcolato dalla ALU come la differenza dei due operandi rs e rt (per realizzare il confronto)

.data

testMem: .word 0x100000

.text

main:

lw **s1**, testMem # carica in s1 il valore di testMem

addi **s0**, **zero**, 1

Sw **s0**, (**s1**) # salva 1 all'indirizzo 0x100000

beq **s1**, **zero**, On # continua, se rotta, la CU memorizza 0

On: # all'ind.testMem

lw **s0**, testMem # carico il contenuto dell'indirizzo
testMed